FINANCIAL® DATA EXCHANGE

Consent API Specification

Version 1.0
May 2025

Legal Notice

Financial Data Exchange, Inc (FDX®) is a standards body and adopts this Consent API Specification for general use among industry stakeholders. Many of the terms, however, are subject to additional interpretations under prevailing laws, industry norms, and/or governmental regulations. While referencing certain laws that may be applicable, readers, users, members, or any other parties should seek legal advice of counsel relating to their particular practices and applicable laws in the jurisdictions where they do business. See FDX's complete Legal Disclaimer located at http://www.financialdataexchange.org for other applicable disclaimers. Copyright © 2025 - FDX - All Rights Reserved.

Revision History

Document Version	Notes	Date
1.0	Initial Document Release from RFC 0355. (Non-incremental update December 2025 for registered trademark symbol, revised document title.)	**May/December 2025**

Contents

1 INTRODUCTION 5

2 CONSENT API USE CASES AND EXCLUSIONS 6

2.1 "CONSENT" VS CONSENTGRANT 6
2.2 CONSENT PARTIES 6
2.3 CONSIDERATIONS FOR DATA ACCESS PLATFORMS 7

3 CONSENT LIFECYCLE 9

3.1 CONSENT GRANT LIFECYCLE 10
3.1.1 Consent API Operations and the User Experience: Summarized Journey 10
3.1.2 Consent API Operations and Data Structures 10

4 CONSENT DATA ENTITIES & TYPES 12

4.1 CONSENT REQUEST 12
4.2 CONSENT GRANT 12
4.3 CONSENT REVOCATION 13

5 CONSENT API OPERATIONS 15

5.1 CREATING CONSENT USING POST /PAR 15
5.1.1 Creating Consent Sequence Diagram & Description of Steps 16
5.1.2 Error Handling 18
5.2 RETRIEVING CONSENT WITH GET /CONSENTS/{CONSENTID} 18
5.2.1 Error Handling 20
5.3 CONSENT REVOCATION WITH `PUT /CONSENTS/{CONSENTID}/REVOCATION` 20
5.3.1 Error Handling 21
5.4 RETRIEVING CONSENT REVOCATION RECORD WITH `GET /CONSENTS/{CONSENTID}/REVOCATION` 22
5.5 RETRIEVE REVOKED CONSENT GRANT FROM `GET /CONSENTS/{CONSENTID}` 22
5.6 CONSENT NOTIFICATIONS TO ENABLE SYNCHRONIZATION 24
5.6.1 Notification Category & Type 24
5.6.2 Notification Payload 24
5.7 FUTURE API OPERATIONS 25

6 RELATIONSHIP BETWEEN THE OAUTH FRAMEWORK AND THE CONSENT FRAMEWORK 26

6.1 FRAMEWORK INTERACTIONS 26
6.2 CONSENT GRANT LIFECYCLE: INTERACTION WITH OAUTH API OPERATIONS & OBJECTS 26
6.2.1 Consent Data Objects 26
6.2.2 Related OAuth Data Objects 27
6.3 FDX BEARER TOKEN OBJECTS 28
6.3.1 Introduction of the `fdxConsentId` Claim 28
6.3.2 Introduction of FDX-Defined Scopes 28
6.3.3 Examples of the `fdxConsentId` Claim and FDX-Defined Scopes in Decoded JWT Bearer Token & Introspection Response 28
6.4 ALIGNMENT OF OAUTH SCOPE TO DATA CLUSTER 29
6.4.1 OAuth Scopes 30
6.4.2 FDX Data Clusters 30
6.4.3 OAuth Scope ↔ FDX Data Cluster Dictionary 31
6.4.4 OAuth Scopes for Notifications APIs 33

6.5 ALIGNMENT OF OTHER MEMBERS OF THE FDX CONSENTGRANT ENTITY WITH OTHER OAUTH CLAIMS 34
6.5.1 Issuer, Subject, Audience, and Client ID Claims 34
6.5.2 Token Lifetime and Other Time-Scopes 34
6.5.3 Consent Grant Revocation 34
6.5.4 Cryptography Standards 34

1 Introduction

This Consent API Specification outlines standardized authorization and consent management methods designed to support clarity, interoperability, and consistency across the Financial Data Exchange (FDX) ecosystem. It specifically addresses the full lifecycle of consent - including creation, synchronization, revocation, and expiration - while facilitating secure, user-permissioned data sharing among data providers, data access platforms, and data recipients.

Currently, this document represents an evolving specification, reflecting ongoing industry collaboration, iterative refinement, and alignment with evolving regulatory frameworks. It is important to note that these consent management methods are NOT part of the current Consumer Standard Data Framework (CSDF) and do NOT impose any conformance obligations or legal requirements. FDX may include refined standards across various authorization and consent management topics in future versions of the CSDF. These topics include, but are not limited to, definitions of data clusters and scopes, mechanisms for consent information exchange, registration processes for Data Recipients, revocation and reauthorization methods, and transmission of signed authorization disclosures.

As the financial data-sharing landscape advances, this Consent API Specification will continue to evolve, guided by industry consensus and regulatory developments. Organizations implementing this specification should anticipate iterative updates and actively participate in the collaborative refinement process facilitated by FDX.

2 Consent API Use Cases and Exclusions

The FDX Consent API enables five primary use cases by specifying API operations and data structures to support:

1. **Consent Creation** — aligned with the Consent Request user flow defined by the FDX User Experience Guidelines resulting in a Consent Grant.
2. **Consent Retrieval** — enabling Consent Synchronization. This is the process by which multiple parties can ensure they have a consistent record of a user's permission for financial data access.
3. **Consent Modification** — enabling changes to the Consent Grant triggered by a user or non-user-facing business logic. Modification is modeled as "revoke and re-issue," i.e. the original Grant is revoked and a new Grant is re-issued in its place with a reference to the original Grant.
4. **Consent Revocation** — aligned with the Consent Revocation user flows defined by the FDX User Experience Guidelines.
5. **Consent Expiration**.

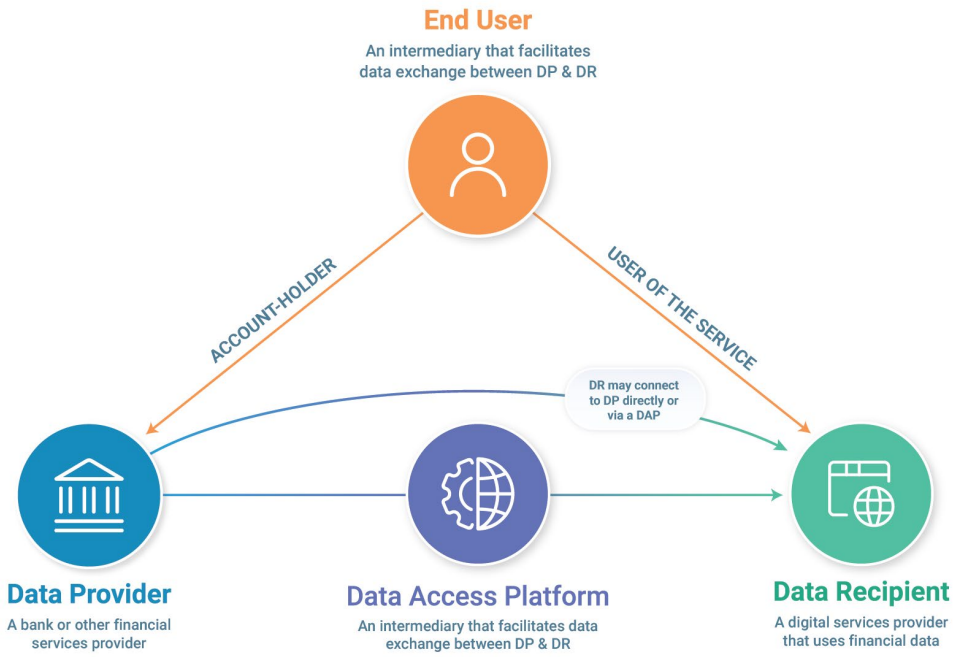
The FDX Consent API does not enforce (or even define) mechanisms for ensuring that data is exchanged in concordance with the user's intents. Furthermore, the FDX Consent API does not define the application or business logic internal to Data Provider and Data Recipient systems and services that align Consent capabilities with data availability or API capabilities.

2.1 "Consent" vs `ConsentGrant`

In an effort to avoid ambiguity, throughout this document, **Consent**, **User Consent**, and **Consent Grant** are used colloquially to refer to the record of the permission granted by an end user for data access. `ConsentGrant` is the specific data entity defined as part of the FDX API specification. A record of User Consent is exchanged between parties using the `ConsentGrant` data entity.

2.2 Consent Parties

A Consent Grant in the FDX framework represents permission to exchange an end user's financial data between data sharing parties. The standard model includes three primary parties: the End User ("EU"), the Data Provider ("DP"), and the Data Recipient ("DR"). However, in some scenarios, a four-party consent model may apply, particularly when parties such as a Data Access Platform ("DAP") facilitate data exchange.



Consent represents the following to each of the three parties:

- To the **Data Recipient** (and **Data Access Platform)**, a Consent Grant is permission to *access* End User's financial data. The DR/DAP's use of this data is generally governed by terms of service with the End User (these terms of service are not incorporated into the FDX Consent framework).
- To the **Data Provider** a Consent Grant is permission to provide access to the Data Recipient for the End User's financial data. A Data Provider generally makes no assertions about Data Recipient's use of the data.
- To the **End User** a Consent Grant is their record that they have given permission for data access. End User should not be expected to disambiguate access vs. use of their financial data.

While the standard API operations outlined in this specification align with a three-party model (End User, Data Provider, and Data Recipient), the presence of a Data Access Platform in a four-party consent model does not inherently alter the protocol-level representation of consent. Instead, the DAP acts as the API client fulfilling the Data Recipient role, effectively facilitating consent exchange on behalf of an authorized third party ("ATP"). Future enhancements to the Consent API will further address DAP involvement.

2.3 Considerations for Data Access Platforms

In the North American ecosystem, DAPs commonly facilitate consented data sharing by serving as the **API client.** Unlike a three-party model, where the DR directly interacts

with the Data Provider, a four-party model allows a DAP to be the API client while retrieving data on behalf of an Authorized Third Party (“ATP”).

Because the DAP is the API client, it assumes the Data Recipient role in API interactions, ensuring that all API requests, authorizations, and data retrieval operations are executed as defined by the FDX specification. The ATP, while benefiting from the data exchange, does not directly interact with the DP’s FDX API but instead relies on the DAP for data access.

While this document primarily describes a three-party model, implementations where a DAP acts as the API client remain fully compatible with FDX’s Consent API structure.

FDX acknowledges the role of DAPs and the four party data sharing model and plans to continue to refine consent models in future iterations.

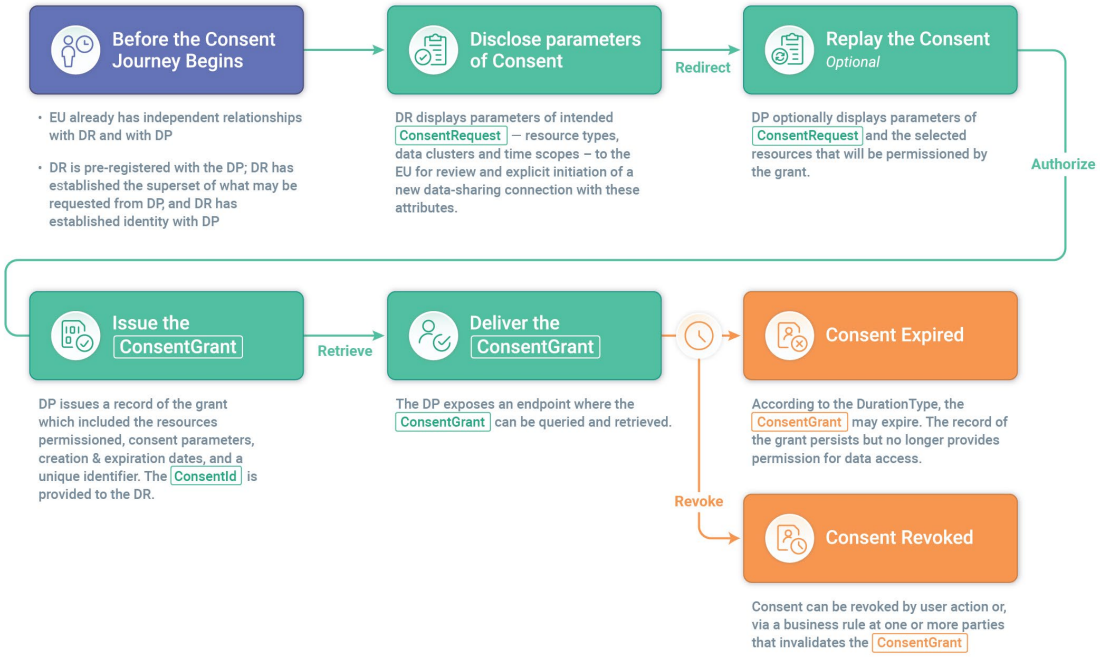
3 Consent Lifecycle

Consent is requested and granted as follows in the three-party model; this user journey through presentation and collection of permission is aligned with the FDX User Experience Guidelines. The FDX Consent framework has a number of preconditions and assumptions:

- Before the consent journey begins, EU has independent relationships with each DR and DP;
- Before the consent journey begins, DR is pre-registered with the DP:
    - DR must establish the superset of what may be requested from DP, and
    - DR must establish identity with DP (both in terms of OAuth client registration and in terms of a bilateral business agreement);
- EU initiates their consent journey from within DR's user experience, per UX Guidelines;
- While DR determines the scope of data access it *seeks* to access from DP, it discloses its intent to the EU;
- EU *actively* authorizes Data Provider to enable Data Recipient's access to End User's data.

3.1 Consent Grant Lifecycle

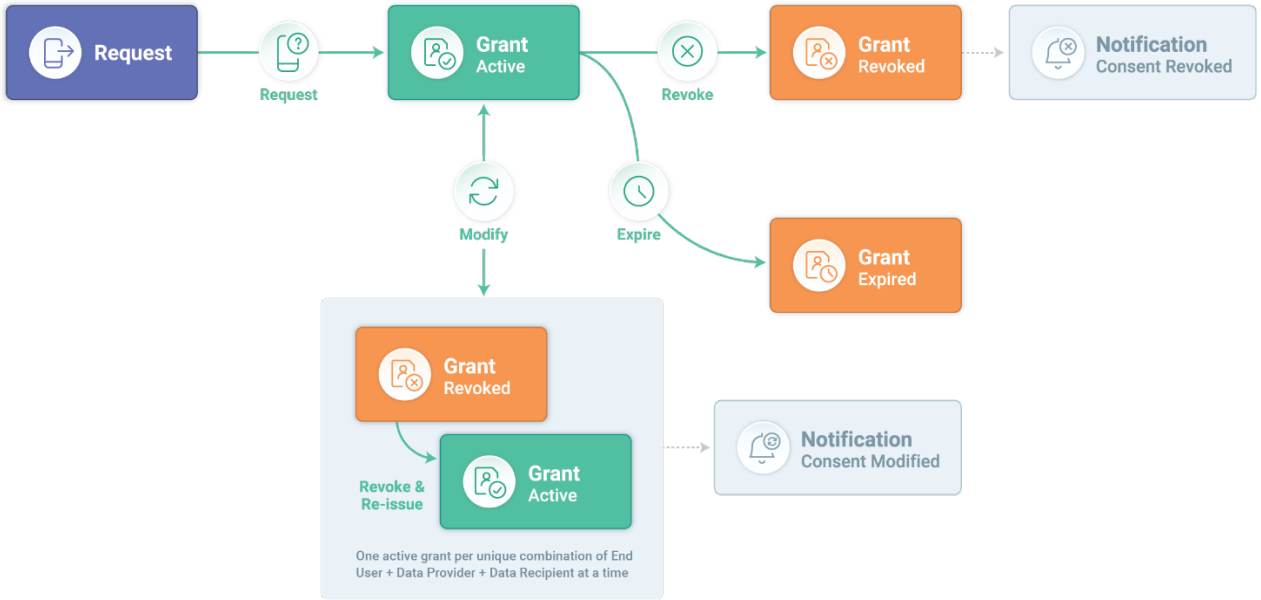
3.1.1 Consent API Operations and the User Experience: Summarized Journey



1. DR presents its Consent Request to the EU, disclosing the scope of desired data access.
2. If accepted by the EU, DR passes its request using the `ConsentRequest` entity within an authorization request to the DP while synchronously redirecting EU to DP.
3. DP authenticates the EU, then "replays" the Consent Request to the EU. The EU may authorize the requested access, "granting consent" for the data exchange.
4. If authorized, Data Provider provides the `ConsentGrant` to the DR.
5. Later, Consent may expire, be revoked operationally or revoked by the EU.

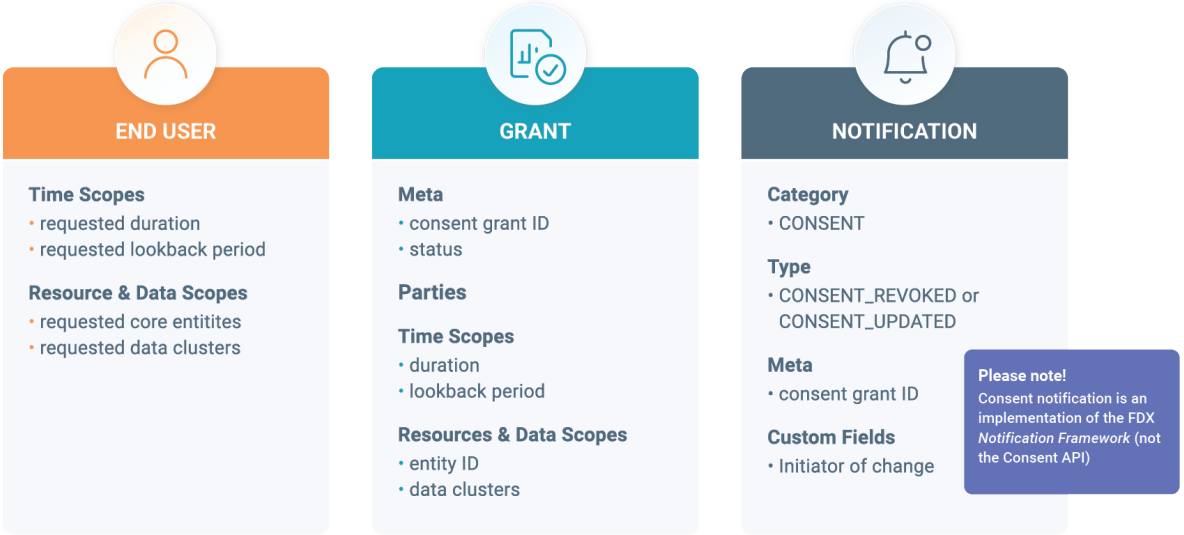
3.1.2 Consent API Operations and Data Structures

As noted above, in section 1 Consent API Use Cases and Exclusions, the Consent API defines API operations and related data structures — primarily request and response payloads — that drive the lifecycle of the Consent Grant. Those lifecycle operations are: **creation** (request and issuance), **retrieval**, **modification**, **revocation**, and **expiration**.



In addition, the framework defines notification payloads used to communicate asynchronously about Consent lifecycle changes using the FDX Notification framework.

4 Consent Data Entities & Types



4.1 Consent Request

The `ConsentRequest` is a data entity consisting of the following scopes, it is transported as a member of an OAuth 2.0 **Rich Authorization Request** (see section **4.1** below).

1. **Time Scopes** — a collection of parameters asserting Duration Type, Duration Period of consent, and Lookback Period which describes a historical period for which data can be retrieved;
2. **Resource Types** — enumeration of the core Resource Types (ie: Accounts, or others such as a Customer Profile or a Document) for which the Data Clusters are requested;
3. **Data Clusters** — enumeration of the Clusters of data elements requested.

The `ConsentRequest` *entity and related types are defined in the FDX API data specification (fdxapi.consent.yaml).*

4.2 Consent Grant

The `ConsentGrant` is a data entity consisting of the following members:

1. **ID** — an identifier for the record, to permit operations such as retrieval; the `consentId` must be unique to the Consent Issuer (a service of the DP) and it must be a URL-safe string;
2. **Consent Status** — enumeration of the current status of the Consent Grant;

3. **Time Scopes** — a collection of parameters asserting Duration Type, Duration Period of consent, Expiration Time, and Lookback Period which describes a historical period for which data can be retrieved, and timestamps asserting the Creation and Expiration time of the Consent Grant;
4. **Parties** — a collection of parameters identifying the Parties (including the legal entity operating branded products or services) in the data exchange.
    - Descriptive information is collected during DR registration at DP, and populated during issuance by Data Provider from its registry;
5. **Resources** — identifiers of the core Resources (ie: Accounts, or others such as a Customer Profile or a Document) for which a set of Data Clusters are permissioned;
6. **Data Clusters** — enumeration of the Clusters of data elements permissioned by this Consent Grant.
7. **Links** — may be used to provide HATEOS links for related records such as revocations of previous Consent Grant(s) governing this EU:DR:DP relationship.

The `ConsentGrant` *entity and related types are defined in the FDX API data specification (fdxapi.consent.yaml).*

4.3 Consent Revocation

There are two additional data structures defined within the FDX Consent framework that facilitate revocation API operations: a **Consent Revocation Record** and **Consent Revocation Request**.

The `ConsentRevocationRequest` is a data entity consisting of the following members:

1. **ID** — an identifier for the record, to permit operations such as retrieval; the `consentId` must be unique to the Consent Issuer (a service of the DP) and it must be a URL-safe string;
2. **Reason** — enumerator of revocation reason (user action or business rule);
3. **Initiator** — contains `ConsentParty` type of the entity initiating revocation.

The `ConsentRevocation` record is a data entity consisting of the following members:

1. **ID** — an identifier for the record, to permit operations such as retrieval; the `consentId` must be unique to the Consent Issuer (a service of the DP) and it must be a URL-safe string;
2. **Consent Status** — enumeration of the current status of the Consent Grant;
3. **Reason** — enumerator of revocation reason (user action or business rule);
4. **Initiator** — contains `ConsentParty` type of the entity initiating revocation;

5. **Timestamp** — recording when the Consent Grant was revoked.

```
{
  "revocations": [
    {
      "status": "REVOKED",
      "reason": "BUSINESS_RULE",
      "initiator": "DATA_PROVIDER",
      "updatedTime": "2023-12-14T15:45:16.987Z"
    }
  ]
}
```

The `ConsentRevocationRequest` and `ConsentRevocation` *entities and related types are defined in the FDX API data specification (fdxapi.consent.yaml).*

5 Consent API Operations

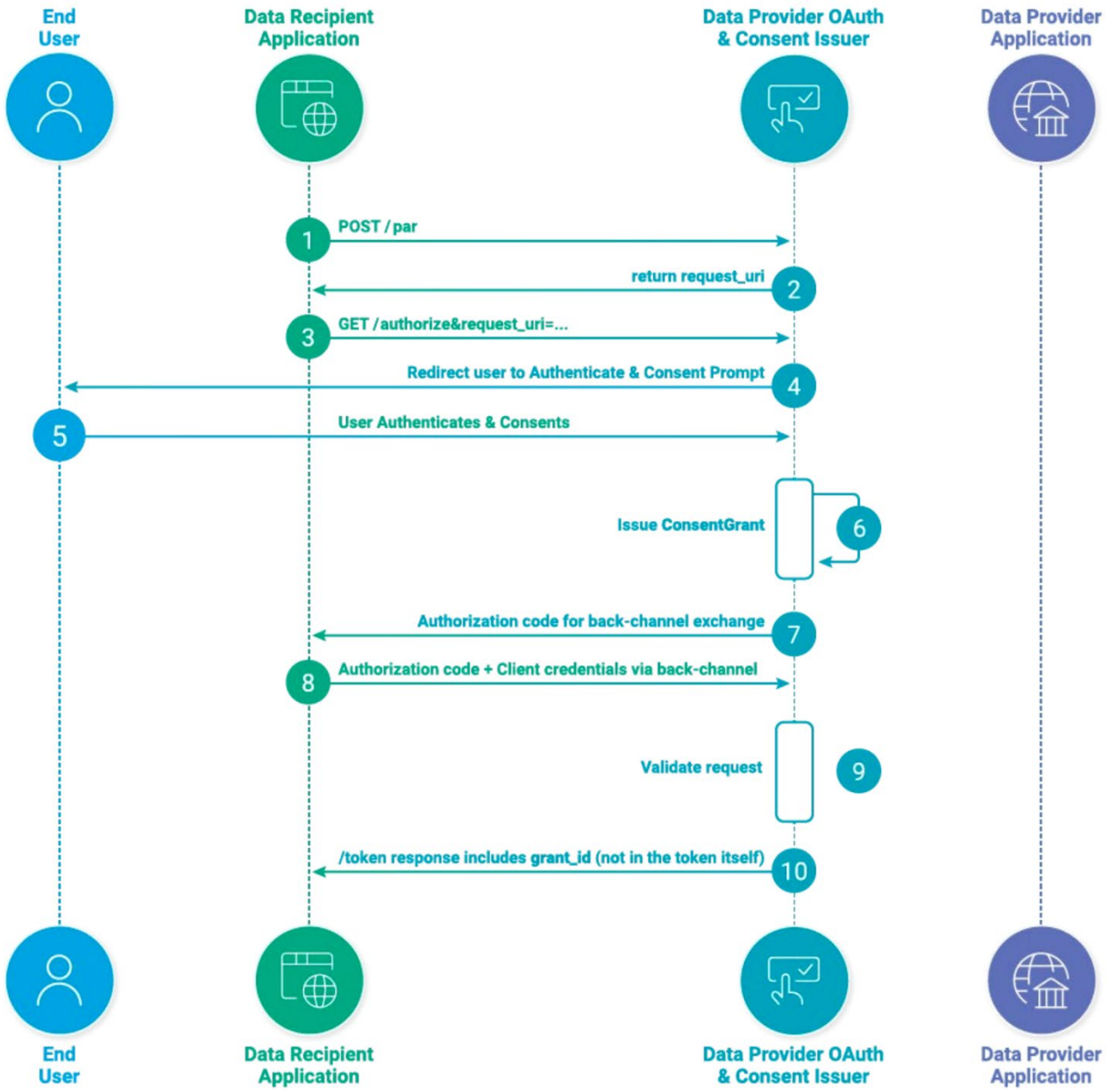
This section specifies a protocol for the DR to make consent requests and for the DP to respond to those requests conveying the user's intent to grant permission for data access. At present, the FDX Consent API defines operations related to: **creation** (request and issuance), **retrieval**, **modification**, **revocation**, and **expiration**.

Not all operations are modeled as direct actions on a `ConsentGrant` data object; the Consent API makes use of two other API frameworks — OAuth 2.0 and FDX Notifications — to transports Consent data. See section 1 for a deeper discussion of the **Relationship between the OAuth Framework and the Consent Framework**.

5.1 Creating Consent using POST /par

The Consent Request is made as part of a three-legged OAuth authorization request using the methods defined by the OAuth 2.0 Pushed Authorization Requests IETF specification. It makes use of the **rich authorization request** protocol as defined by the OAuth 2.0 Rich Authorization Requests IETF specification. (In the event of discrepancies, RAR and PAR specifications prevail.)

5.1.1 Creating Consent Sequence Diagram & Description of Steps



Step 1 — DR initiates a POST request to DP's `POST /par` endpoint using the Authorization Request (PAR) method

DR must be authenticated with methods allowed by the FDX API Security Model, Part 1 FDX Security Profile for OAuth client authentication. The authorization_details request parameter MUST contain a JSON-formatted object with two members in compliance with the RAR format specified by RAR memo, section 2:

1. `type` parameter with value `fdx_v1.0`, and
2. `consentRequest` parameter containing a valid `ConsentRequest` entity;

For example:

```
{
  "authorization_details": [
    {
      "type": "fdx_v1.0",
      "consentRequest": {
        "durationType": "ONETIME",
        "lookbackPeriod": 60,
        "resources": [
          {
            "resourceType": "ACCOUNT",
            "dataClusters": [
              "ACCOUNT_DETAILED",
              "TRANSACTIONS",
              "STATEMENTS"
            ]
          }
        ]
      }
    }
  ]
}
```

Step 2 — On success, DP responds with a `201 Created` HTTP response code and JSON response

Endpoint behavior and responses defined in detail by [OAuth 2.0 Pushed Authorization Requests](), § 2.

Step 3 — DR uses the returned `request_uri` to build its subsequent request to `GET /authorize`

Endpoint behavior and responses defined in detail by [OAuth 2.0 Pushed Authorization Requests](), § 4.

Steps 4, 5 — User authentication, consent and authorization

Defined in detail by FDX UX Guidelines. DP requires input and explicit authorization from an authenticated EU to issue the `ConsentGrant`.

Step 6 — If successful, DP issues the `ConsentGrant` including a unique identifier for the record, `consentId`

Steps 7, 8, 9 — If successful, DP returns a `302 Found` HTTP response code, and an authorization `code` for DR to build its request to `POST /token`

Endpoint behavior and responses defined in detail by [OAuth 2.0 Authorization Framework](), § 5.

Step 10 — Successful token response will include the `grant_id` parameter containing `consentId` in addition to token response contents

5.1.2 Error Handling

Errors for `GET /authorize` MUST follow OAuth 2.0 standards: In all cases, DP responds with `302 Found` HTTP code and informs the client of the error by adding error parameters to the query component of the redirection URI using the `application/x-www-form-urlencoded` format.

In instances of mismatch between scopes requested by the DR in the ConsentRequest and the pre-registered scopes (e.g.: invalid Data Clusters, Resource Types, or Duration), service MUST respond with `invalid_scope` error code.

For example:

```
HTTP/1.1 302 Found
Location: https://client.example.com/cb?error=invalid_scope
```

Other errors (eg: invalid client, failed user authentication, or access denied) MUST be handled per [OAuth 2.0 Authorization Framework](), § 5.2.

5.2 Retrieving Consent with GET /consents/{consentId}

Consent Grant retrieval is used to synchronize Consent across parties; *retrieval requests may originate from any party* and may occur ad hoc, on a scheduled basis, or after a lifecycle event (i.e. Consent Creation or Consent Revocation) in response to a notification.

API client initiates the operation by calling the `GET /consents/{consentId}`.

The `consentId` was delivered in successful `POST /token` response during Consent Creation. The client must be authenticated with methods allowed by the FDX API Security Model, Part 1 FDX Security Profile.

Server responds with the `ConsentGrant` in the JSON response body.

For example:


```
{
  "id":"9585694d3ae58863",
  "status":"ACTIVE",
  "parties":[
    {
      "name":"Seedling App",
      "homeUri":"https://www.seedling.com",
      "logoUri":"https://www.seedling.com/assets/seedling-
logo.png",
      "legalEntityName":"Oak Tree Holdings, Inc",
      "legalEntityIdentifier":"5493001052I34KDC1O18"
    },
    {
      "name":"Midwest Primary Bank, NA",
      "homeUri":"https://www.midwest.com",
      "logoUri":"https://www.midwest.com/81d88112572c.jpg",
      "registryName":"GLEIF",
      "registeredEntityName":"Midwest Primary Bank, NA",
      "registeredEntityIdentifier":"549300ATG070THRDJ595"
    }
  ],
  "createdTime":"2021-07-03T21:28:10.375Z",
  "updatedTime":"2021-12-14T15:45:16.987Z",
  "durationType":"TIME_BASED",
  "lookbackPeriod":"60",
  "resources":[
    {
      "resourceType":"ACCOUNT",
      "resourceId":"b14e1e714693bc00",
      "dataClusters":[
        "ACCOUNT_DETAILED",
        "TRANSACTIONS",
        "STATEMENTS"
      ]
```

```
    }
  ],
  "links": {
    "revocation": {
      "href": "/consents/9585694d3ae58863/revocation",
      "action": "GET"
      }
  }
}
```

The `ConsentGrant` *entity and related types are defined in the FDX API data specification (fdxapi.consent.yaml).*

5.2.1 Error Handling

Server MUST handle two categories of errors for `GET /consents/{consentId}`: unauthorized requests and invalid requests.

If a client attempts to retrieve a `ConsentGrant` with invalid credentials, server MUST respond with a `401 Unauthorized` HTTP code, further error description is not required or recommended.

If a known client attempts to retrieve a `ConsentGrant` they are either not permitted to access or that doesn't exist, server MUST respond with a `404 Not Found` description is not required or recommended.

5.3 Consent Revocation with `PUT /consents/{consentId}/revocation`

Revoking Consent is classified as a write operation on the `ConsentGrant` data object. The result of Consent revocation is a status change on the `ConsentGrant` from `ACTIVE` to `REVOKED`. Revocation is complete and cannot be reversed; once revoked, access to all resources and data clusters by data recipient is no longer permitted. To reestablish access, a new `ConsentGrant` must be created.

Requestor initiates a PUT request to DP's `PUT /consents/{consentId}/revocation` endpoint

The `consentId` was delivered in successful `POST /token` response during Consent Creation. The client must be authenticated with methods allowed by the FDX API Security Model, Part 1 FDX Security Profile. The request body consists of the reason for the revocation and the party initiating the revocation.

Consent can be revoked by user action, such as in the following examples:

- User “disconnects” an FI (Data Provider) from an application (Data Recipient), or
- User ends their relationship with a given application (Data Recipient).

The following revocation payload would be used:

```
{
  "reason": "USER_ACTION",
  "initiator": "INDIVIDUAL"
}
```

Or consent can be revoked via a business rule at one or more parties that invalidates the Consent, such as in the following example:

- An FI (Data Provider) and application (Data Recipient) discontinue their data access relationship.

The following revocation payload would be used:

```
{
  "reason": "BUSINESS_RULE",
  "initiator": "DATA_RECIPIENT"
}
```

Consent Issuer responds with `204` (no response body).

`204` SHOULD be used to indicate that the revocation request has been received and will be processed, but may not be immediately revoked

- if revocation is asynchronous, issuer MAY follow-up with a notification to signal a completed revocation.

The `ConsentRevocationRequest` *entity and related types are defined in the FDX API data specification (fdxapi.consent.yaml).*

5.3.1 Error Handling

Errors for `PUT consents/{ConsentId}/revocation` MUST follow FDX API error standards:

- `400 Bad Request` SHOULD be used with an improperly formatted request
- `401 Unauthorized` SHOULD be used if the error results from inadequate authentication
- `403 Forbidden` SHOULD be used if the error results from inadequate authorization

- `404 Not Found` SHOULD be used if the specified `ConsentGrant` is not applicable to the validly authenticated client
- `409 Conflict` SHOULD be used if a request is made to revoke an already revoked or expired `ConsentGrant`

5.4 Retrieving Consent Revocation Record with `GET /consents/{consentId}/revocation`

A requestor or other permissioned party may wish to display the history of a Consent Grant to the End User, following revocation. This record of this change is available to Data Recipient or Data Access Platform requestor with necessary permission to retrieve the Consent Grant.

Requestor initiates a GET request to DP's `GET /consents/{ConsentId}/revocation` endpoint.

The client must be authenticated with methods allowed by the FDX API Security Model, Part 1 FDX Security Profile.

Consent Issuer responds with an array of timestamped records of all revocations:

```
{
  "revocations": [
    {
      "status": "REVOKED",
      "reason": "BUSINESS_RULE",
      "initiator": "DATA_PROVIDER",
      "updatedTime": "2023-12-14T15:45:16.987Z"
    }
  ]
}
```

The `ConsentRevocation` entity and related types are defined in the FDX API data specification (fdxapi.consent.yaml).

5.5 Retrieve Revoked Consent Grant from `GET /consents/{consentId}`

Any permissioned party may request the record of revoked Consent Grant.

API client initiates the operation by calling the `GET /consents/{consentId}`.

The client must be authenticated with methods allowed by the FDX API Security Model, Security Profile.

Server responds with the `ConsentGrant` in the JSON response body, status = `REVOKED`.

For example:


```
{
  "id":"9585694d3ae58863",
  "status":"REVOKED",
  "parties":[
    {
      "name":"Seedling App",
      "homeUri":"https://www.seedling.com",
      "logoUri":"https://www.seedling.com/assets/seedling-
logo.png",
      "legalEntityName":"Oak Tree Holdings, Inc",
      "legalEntityIdentifier":"5493001052I34KDC1O18"
    },
    {
      "name":"Midwest Primary Bank, NA",
      "homeUri":"https://www.midwest.com",
      "logoUri":"https://www.midwest.com/81d88112572c.jpg",
      "registryName":"GLEIF",
      "registeredEntityName":"Midwest Primary Bank, NA",
      "registeredEntityIdentifier":"549300ATG070THRDJ595"
    }
  ],
  "createdTime":"2021-07-03T21:28:10.375Z",
  "updatedTime":"2021-12-14T15:45:16.987Z",
  "durationType":"TIME_BASED",
  "lookbackPeriod":"60",
  "resources":[
    {
      "resourceType":"ACCOUNT",
      "resourceId":"b14e1e714693bc00",
      "dataClusters":[
        "ACCOUNT_DETAILED",
```

```
        "TRANSACTIONS",
        "STATEMENTS"
      ]
    }
  ],
  "links": [
    {
      "href": ".../consents/9585694d3ae58863/revocation",
      "action": "GET",
      "rel": "revocation"
    }
  ]
}
```

The `ConsentGrant` *entity and related types are defined in the FDX API data specification (fdxapi.consent.yaml).*

5.6 Consent Notifications to Enable Synchronization

The FDX Consent framework does not define direct mechanisms (API operations or data structures) to modify a Consent Grant. However, the FDX Notifications framework may be used to publish event notifications alerting other parties of modifications and revocations. To support use of these event notifications for Consent Synchronization, Consent-specific structures have been defined.

5.6.1 Notification Category & Type

Consent Notifications consist of the following members, in compliance with the FDX Notifications data-structure:

1. **Category** — use the enumeration `CONSENT`;
2. **Type** — one of `CONSENT_UPDATED` or `CONSENT_REVOKED`;

5.6.2 Notification Payload

The notification payload for Consent notifications consist of the following members, in compliance with the FDX Notifications data-structure:

1. **ID Type** — use the enumeration `CONSENT`;
2. **ID** — use `consentId`;
3. **Custom Fields** —
    - **Name** — use the enumeration `INITIATOR`;

- **Value** — one of `INDIVIDUAL` (used when the End User initiated the change), `DATA_RECIEPIENT` or `DATA_PROVIDER` (when the change is initiated by business logic) to denote the party initiating the modification or revocation.

For example:

```
{
  "category": "CONSENT",
  "type": "CONSENT_REVOKED"
  "notificationPayload": {
    "id": "9585694d3ae58863"
    "idType": "CONSENT",
    "customFields": {
      "name": "INITIATOR",
      "value": "INDIVIDUAL"
    }
 }
```

`ConsentId` *and* `PartyType` *types are defined in the FDX API data specification (fdxapi.consent.yaml).*

5.7 Future API Operations

A number of additional Consent API operations have been considered and may be included in future versions of the specification. A non-exhaustive list of those considered includes API operations for:

- Consent Grant discovery, for example retrieving all `ConsentIds` for a given API client or Customer,
- Additional mechanisms to enable Consent modification, and
- Additional mechanisms to enable Consent portability.

6 Relationship between the OAuth Framework and the Consent Framework

6.1 Framework Interactions

The Consent framework defines a fine-grained approach to user-driven data-sharing authorization within the FDX ecosystem. It extends typical OAuth implementations used for API authorization offering the ability to synchronize data elements of Consent beyond what is specified by OAuth objects. This additional details afford two major benefits to the financial data-sharing ecosystem:

1. Consent synchronization can be supported both between an entity's internal services, or amongst parties to Consent (data providers, data recipients, data access platforms, and end users) using the same protocols; and
2. OAuth implementations that may not support granular authorization data may be employed in tandem with the Consent framework to provide implementation optionality without sacrificing robustness.

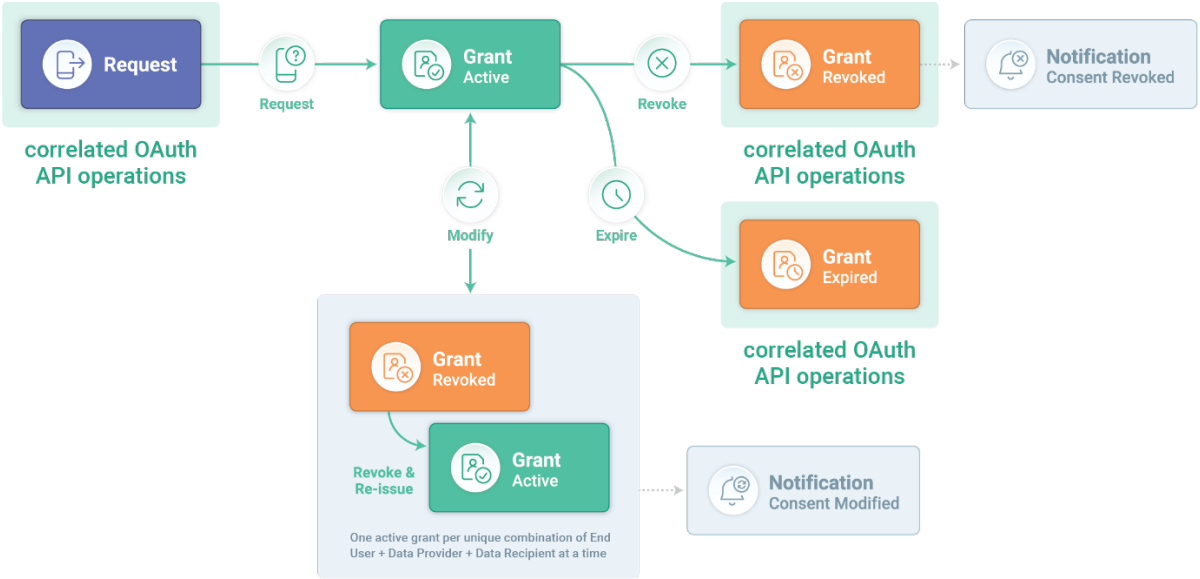
The Consent APIs and data structures do not seek to *replace* OAuth API operations or data structures, rather the framework is intended to normalize the Consent data lifecycle and synchronization across varying OAuth implementations.

6.2 Consent Grant Lifecycle: Interaction with OAuth API Operations & Objects

The primary data entity specified by the FDX Consent framework is the Consent Grant; the Consent API defines API operations and related data structures — request and response payloads — that drive the lifecycle of the Consent Grant. Those lifecycle operations are: **creation** (aka issuance), **retrieval**, **modification**, **revocation**, and **expiration**.

6.2.1 Consent Data Objects

A **Consent Grant**, FDX core entity `ConsentGrant`, is the demonstrable result of a completed **Consent Request**. (`ConsentGrant` is an *exchange* format and not intended to define all parameters required for internal record-keeping.) FDX core entity `ConsentRequest`, defines a standardized set of request parameters aligned with the recommended user-facing details present in the Create Consent user journey as defined by the FDX User Experience Guidelines. The `ConsentRequest` is passed within a Rich Authorization Request object, described below.



The Consent framework also specifies OAuth API operations that are correlated with Consent Grant state changes. Depending on implementation, notifications may also prompt OAuth operations.

6.2.2 Related OAuth Data Objects

There are three related data structures defined by the OAuth Framework that have defined interactions with the FDX Consent framework:

1. **Rich Authorization Request** as defined by the [OAuth 2.0 Rich Authorization Requests]() IETF specification. This request object is passed in the `authorization_details` parameter of a **GET /authorize** request. This IETF specification also defines the relationship between the `authorization_details` parameter and the `scope` parameter:

"`authorization_details` and `scope` can be used in the same authorization request for carrying independent authorization requirements.

"Combined use of `authorization_details` and `scope` is supported by this specification in part to allow existing OAuth-based applications to incrementally migrate towards using `authorization_details` exclusively. It is RECOMMENDED that a given API use only one form of requirement specification."

https://datatracker.ietf.org/doc/html/draft-ietf-oauth-rar#name-relationship-to-scope-param

The rich authorization request is used in conjunction with a **POST /par** operation compliant with [OAuth 2.0 Pushed Authorization Requests]() IETF specification to request Consent for data-sharing.

2. **Bearer Token Profile** as defined by the FDX JWT Profile, `JwtProfile`, which is compliant with the IETF specification for JSON Web Token (JWT).
3. **Bearer Token Introspection Response**, `FdxTokenIntrospectionResponse`, which is compliant with the IETF specification for OAuth 2.0 Token Introspection.

6.3 FDX Bearer Token Objects

A JWT-compliant OAuth `access_token` may be used to transport some details of the Consent Grant: the unique identifier (Consent ID) within a private claim; and FDX-defined scopes may be declared as optional members of the Authorization Claim, aka `scope` parameter. No changes to the token issuance flows, grant types, or other OAuth API operations are intended.

6.3.1 Introduction of the `fdxConsentId` Claim

The authorization server may optionally convey to the resource server FDX `consentId` within the `access_token` included in a Private Claim claim named `fdxConsentId`. The `fdxConsentId` claim contains a string; this member is the identifier of the current user's active `ConsentGrant`.

6.3.2 Introduction of FDX-Defined Scopes

FDX scopes are constructed as follows:

- Scopes will prepend an `fdx` prefix to encourage interoperability with other standards and avoid potential scope name collisions.
- Scopes will append an operation identifier to explicitly declare what actions are authorized.

Most FDX-defined scopes align with Consent Grant Data Clusters, see below.

6.3.3 Examples of the `fdxConsentId` Claim and FDX-Defined Scopes in Decoded JWT Bearer Token & Introspection Response

JWT Profile

```
{
 "iss": "https://authorization-server.example.com/",
 "sub": "5ba552d67",
 "aud": "https://rs.example.com/",
 "exp": 1639528912,
 "iat": 1618354090,
```

```
 "jti" : "dbe39bf3a3ba4238a513f51d6e1691c4",
 "client_id": "s6BhdRkqt3",
 "scope": "fdx:transactions:read fdx:statements:read openid
offline_access",
 "fdxConsentId": "28920138920832"
}
```

Introspection Response

These data elements may optionally be used in the token introspection response, extending RFC 7662: OAuth 2.0 Token Introspection, for use in implementations that utilize opaque access tokens (and other use cases at the implementer's discretion).

```
{
  "active": true,
  "iss": "https://authorization-server.example.com/",
  "sub": "5ba552d67",
  "aud": "https://rs.example.com/",
  "exp": 1639528912,
  "iat": 1618354090,
  "jti" : "dbe39bf3a3ba4238a513f51d6e1691c4",
  "client_id": "s6BhdRkqt3",
  "scope": "fdx:transactions:read fdx:statements:read openid
offline_access",
  "fdxConsentId": "28920138920832"
}
```

6.4 Alignment of OAuth Scope to Data Cluster

Definition of FDX-defined OAuth scopes complements Consent Creation API request flow, OAuth Client Registration, and recommended Security Profile specified by FDX. Scope alignment seeks to "close the gap" in implementing FDX Consent Framework via OAuth. This "dictionary" unifies enumerated data elements across:

- **OAuth Client Registration** — defining the total universe of data-sharing available between a DP and DR
- **Consent Request and Grant** — defining the specific data clusters and resource types desired for a particular user's connectivity
- **Authorization Token Format & Lifecycle** — defining the scope of authorization permissioned by the user for data exchange via API operation

The following is intended to help answer: *What are the differences between an OAuth Scope and a Data Cluster used within the Consent Grant record? Why do they have different formats?*

6.4.1 OAuth Scopes

A `scope` is a string defined by an authorization server that specifies what data may be accessed and it what manner it may be accessed, i.e. which API operation(s) are permitted. IETF-defined scopes (e.g., `offline_access` specified by the OpenID Connect Specification), privately-defined scopes, and FDX-defined scopes may be intermingled and used within any of OAuth API requests or credential objects.

"The value of the scope parameter is expressed as a list of space-delimited, case-sensitive strings. The strings are defined by the authorization server."
– https://datatracker.ietf.org/doc/html/rfc6749#section-3.3

```
{
"iss": "https://authorization-server.example.com/",
"Sub": "5ba552d67",
"aud": "https://rs.example.com/",
"exp": 1639528912,
"iat": 1618354090,
"jti" : "dbe39bf3a3ba4238a513f51d6e1691c4",
"client_id": "s6BhdRkqt3",
"scope": "fdx:transactions:read fdx:statements:read openid offline_access",
"fdxConsentId", "28920138920832"
}
```

FDX-defined scopes IETF-defined scopes

FDX-defined scopes use a name-spacing mechanism to distinguish them and explicitly tie their usage to this specification. FDX scopes are constructed as follows: **FDX namespace** + separator + **Data Cluster** + separator + **Operation verb**:

```
fdx:statements:read
```

namespace data cluster operation

6.4.2 FDX Data Clusters

Within the FDX API and data structure specifications, including the `ConsentGrant` entity, data clusters are an enumerated value intended to explicitly collect related data elements. The FDX specification uses SCREAMING_SNAKE case for enums. The data cluster does *not* specify the operation performed on that data.

6.4.3 OAuth Scope ↔ FDX Data Cluster Dictionary

Please see FDX UX Guidelines for other details about Data Clusters including required data elements.

FDX Scope	*API Operation*	*Data Cluster*	*Description*
`fdx:accountbasic:read`	read	`ACCOUNT_BASIC`	Authorizes retrieval of account identifying information
`fdx:accountdetailed:read`	read	`ACCOUNT_DETAILED`	Authorizes retrieval of account summary information
`fdx:balances:read`	read	`BALANCES`	Authorizes retrieval of account balances
`fdx:bills:read`	read	`BILLS`	Authorizes retrieval of payments due on account
`fdx:customercontact:read`	read	`CUSTOMER_CONTACT`	Authorizes retrieval of account contact details
`fdx:customerpersonal:read`	read	`CUSTOMER_PERSONAL`	Authorizes retrieval of sensitive personal information
`fdx:images:read`	read	`IMAGES`	Authorizes retrieval of check images

FDX Scope	API Operation	Data Cluster	Description
`fdx:investments:read`	read	`INVESTMENTS`	Authorizes retrieval of high-level view of total investments, including balances and asset breakdowns as well as access to detailed information on investment holdings, including stocks, securities, and their current market values.
`fdx:paymentsupport:read`	read	`PAYMENT_SUPPORT`	Authorizes retrieval of full account information for payments
`fdx:rewards:read`	read	`REWARDS`	Authorizes retrieval of rewards program information
`fdx:scheduledpayments:read`	read	`SCHEDULED_PAYMENTS`	Authorizes retrieval of scheduled payment information
`fdx:socialsecurity:read`	read	`SOCIAL_SECURITY_NUMBER`	Authorizes retrieval of social security number

FDX Scope	API Operation	Data Cluster	Description
`fdx:statements:read`	read	`STATEMENTS`	Authorizes retrieval of account statements
`fdx:tax:read`	read	`TAX`	Authorizes retrieval of tax form data
`fdx:termsconditions:read`	read	`TERMS_AND_CONDITIONS`	Authorizes retrieval of account terms and conditions
`fdx:transactions:read`	read	`TRANSACTIONS`	Authorizes retrieval of transaction data
`fdx:transfers:read`	read	`TRANSFERS`	Authorizes retrieval of transfer information
`fdx:transfers:write`	write	`TRANSFERS`	Authorizes creation of transfer information

6.4.4 OAuth Scopes for Notifications APIs

These scopes describe authorization for non-user facing functionality — notifications, aka “webhooks” — and are not aligned with Data Clusters.

FDX Scope	API Operation	Data Cluster	Description
`fdx:notifications:subscribe`	write	N/A	Authorizes creation of a new notification subscription
`fdx:notifications:publish`	write	N/A	Authorizes publishing of a new notification

6.5 Alignment of Other Members of the FDX `ConsentGrant` Entity with Other OAuth Claims

Beyond the `scope` parameter in the FDX JWT Profile, implementers may choose to align other claims with FDX data elements. Data format of all the following parameters are governed by IETF specification JSON Web Token (JWT). Some of the parameters described are governed by the FDX Security Profile; non-normative recommendations are also included. Noted below as applicable.

6.5.1 Issuer, Subject, Audience, and Client ID Claims

The following claims can be used to relate an `access_token` to Consent data entities.

1. **Issuer** — may be used to identify the relevant Consent party (Data Provider).
2. **Subject** — may be used to identify the end user associated with the `access_token`.
3. **Audience** — may be used to identify the relevant Consent party (Data Recipient).
4. **Client ID** — may be used to identify the relevant OAuth Client if not the same entity as the audience (intermediary).

6.5.2 Token Lifetime and Other Time-Scopes

The following claims describe time-scopes of the token **not** the related Consent Grant. Usage and value format (`NumericDate`) as specified by IETF.

1. **Expiration** — recommended. `access_tokens` have a maximum lifetime of 900 seconds as required by FDX Security Profile.
2. **Issued-At** — recommended.
3. **Not-Before** — optional.

6.5.3 Consent Grant Revocation

Upon revocation, any `access_tokens` associated with the Consent Grant shall be revoked immediately.

6.5.4 Cryptography Standards

JWT cryptography standards are defined by the FDX Security Profile.